

6-4 (1) 图的应用

- 1.最小生成树

- 概念

- 生成树：包含图中所有顶点的一个极小连通子图
 - 最小生成树：各边代价之和最小的那棵生成树
 - 生成的边的数量一定是 $n-1$
 - 若原图存在权值相同的边，那么该图的最小生成树可能不唯一
 - 若原图的边权值互不相等。那么该图的最小生成树唯一
 - 最小生成树中所有边的权值之和是所有生成树中最小的

- Prim算法

- 算法流程

- 1.初始时任取一顶点加入树 T
 - 2.选择一个与当前 T 中顶点集合距离最近的顶点，并将其顶点和相应边加入 T ，每次操作后 T 的顶点数和边数都加一
 - 3.直到所有顶点加入 T ，此时有 $n-1$ 条边

- Kruskal算法(适用于边稀疏，顶点多的图)

- 算法流程

- 1.初始时为只有 n 个顶点而无边的非连通图 T ，每个顶点自成一个连通分量
 - 2.按照边的权值从小到大的顺序，选择当前未被选取且权值最小的边。若该边依附的顶点落在 T 中不同的连通分量上，则将此边加入 T ，否则舍弃此边而选择下一条
 - 3.直到 T 中所有顶点都在一个连通分量上

- 2.最短路径

- 概念

- 在图中以最小代价从一个节点到达另一个节点

- Dijkstra算法（求某个源点到其余各顶点的最短路径）

- 算法流程

- 1.初始化三个数组信息
 - 2.观察 $final[]$ 数组，若 $final[i]=0$ 且 $dist[i]$ 最小，则将 $final[i]$ 改为1，收录该顶点 v_i 到最优路径集合中
 - 3.对 v_i 的邻接点 v_j 做如下判断：如果 $dist[i]+<v_i,v_j>$ 的权值 $<dist[j]$ ，则更新 $dist[j]$ 与 $path[j]$

- Floyd算法（求每一对顶点之间的最短路径）

- 算法流程

- 1.初始化矩阵

- 2.选择顶点*i*作为中转点，寻找更短路径，替换矩阵
- BFS方法（仅非带权图）
 - 代码

```
bool visited[MVNum];
void BFS_distance(Graph G, int v) {
    int i, u, w;
    int d[G.vexnum];
    for (i = 0; i < G.vexnum; i++) {
        visited[i] = false;
        d[i] = ∞;
    }
    visited[v] = true;
    d[v] = 0;
    InitQueue(Q);
    EnQueue(Q, v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q, u);
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w)) {
            if (!visited[w]) {
                visited[w] = true;
                d[w] = d[u] + 1;
                EnQueue(Q, w);
            }
        }
    }
}
```